ENGR 102 Summer Bridge

Day 13 Instructor Key

Loop Design, Break, Continue, and Nested Tracing

Wednesday, July 22, 2026 • 50 points

Name		UIN		Section	

Boolean-operator convention. Python operators appear as `and`, `or`, and `not`. Their lowercase spelling is required in code.

Daily target — increasing independence

Select skip, stop, and reset behavior while tracing repeated work inside repeated work.

Allowed tools	Prior tools plus break, continue, and nested for-loop tracing
Support supplied	Behavioral requirements and a sample input stream are supplied.
You must decide	Loop form, placement of skip and stop decisions, state updates, and reset scope.
Scaffold level	Requirements only; students select the control-flow pattern.

Scoring and completion standard — 50 points

Evidence	Points	Secure work shows
Integrated trace	10	intermediate state, condition results, and exact output
Diagnosis and repair	8	actual, intended, cause, repair, and a boundary test when requested
Full program and tests	32	coherent executable logic, required output, and defended tests

Independence standard. You may use the supplied requirements and examples, but you must produce one connected solution. A collection of disconnected correct lines is not yet a complete program.

Begin with the trace, then use its evidence habits when you construct.

Task 1. Integrated trace — 10 points

Trace nested repetition while distinguishing a skipped pass, an early stop, and a per-station reset.

```
overall = 0

for station in range(1, 3):
    station_total = 0
    for reading in [0, station + 1, 5]:
        if reading == 0:
            continue
        if reading == 5:
            break
        station_total = station_total + reading
    overall = overall + station_total
    print(station, station_total)

print(overall)
```

Your task

1. Give a complete reached-pass ledger that identifies the continue, break, station_total reset, and overall update.
2. Write every output line and explain why the inner break does not end the outer loop.

Answer and explanation

1. Station 1 resets station_total to 0. Reading 0 continues; reading 2 is added; reading 5 breaks the inner loop. The station total is 2 and overall becomes 2. Station 2 resets station_total to 0. Reading 0 continues; reading 3 is added; reading 5 breaks the inner loop. The station total is 3 and overall becomes 5.
2. Exact output: 1 2; 2 3; 5. break ends only the innermost loop that contains it, so the outer station loop advances from station 1 to station 2.

A final answer without the requested state evidence cannot earn full trace credit.

Task 2. Diagnose and repair — 8 points

The intent is to print every odd integer through 5 and then stop. The current order omits 5.

```
value = 0
while value < 8:
    value = value + 1
    if value % 2 == 0:
        continue
    if value == 5:
        break
    print(value)
```

Your task

1. Give actual versus intended output, identify the ordering cause, and write a minimal repair that still stops at 5.

Answer and explanation

1. Actual output is 1 and 3. Intended output is 1, 3, and 5. The break occurs before print when value is 5. Minimal repair: move print(value) before the if value == 5 test, then keep break after the print. Even values still continue before printing.

Debugging evidence is complete only when the repair is tied to the stated defect and an expected result.

Task 3. Construct a complete program — 32 points across Pages 4-5

Program specification

- Repeatedly read an integer inspection code.
- Code 0 ends normal input. A negative code is ignored and processing continues.
- A code of 90 or greater is a shutdown code: stop immediately without counting or adding it.
- Each positive code below 90 is accepted. Count accepted codes and add their values.
- After the loop, print accepted_count and total on separate lines.

Answer and explanation

```
accepted_count = 0
total = 0

while True:
    code = int(input())
    if code == 0:
        break
    if code < 0:
        continue
    if code >= 90:
        break
    accepted_count = accepted_count + 1
    total = total + code

print(accepted_count)
print(total)
```

The loop reads once at the top of every pass. Because no input update is placed after continue, the next pass naturally returns to input instead of repeating stale state.

The normal sentinel and shutdown condition both use break but have different meanings. Neither contributes to the count or total.

Task 3 continued. Test and defend the program **included in 32 points**

Expected tests and results

1. Inputs 12, -3, 25, 99, 40 produce accepted_count 2 and total 37; 40 is never read after shutdown.
2. Inputs -5, 8, 0 produce 1 and 8. Immediate 0 produces 0 and 0.

Scoring guidance

6 input/loop architecture; 4 normal sentinel; 4 continue case; 4 shutdown break; 6 state updates; 3 output; 5 tests and control-flow explanation.

Accept alternative correct code when it obeys the stated tool boundary, handles the required cases, and produces the required output. All blue text is answer or scoring guidance.

VIP Lab Alignment

Bridge Day 13 • Contract c821cfcf7189

This page adds a current lab handoff while preserving the workbook's independence-ramp tasks.

Why this page is here

VIP Lab Day(s): 5 **Activities:** 18.18, 18.19, 18.20, 18.21, 18.22

Select a loop form from the requirement, then complete an entire sample/transfer pair without an ordered algorithm being supplied.

Open these current notebook cells

Activity / stable cells	Notebook work	Evidence to carry forward
18.18 `18-18-work` / `18-18-transfer`	**Multiples In Range**	Choose inclusive range bounds.
18.19 `18-19-work` / `18-19-transfer`	**Countdown To Multiple**	Write a while condition from a divisibility goal.
18.20 `18-20-work` / `18-20-transfer`	**Smallest Value**	Initialize comparison state from real data.
18.21 `18-21-work` / `18-21-transfer`	**Cooldown Sequence**	Track a current value and a stored sequence.
18.22 `18-22-work` / `18-22-transfer`	**Rounds To Clear Queue**	Model repeated queue reduction.

Readiness check before you code

1. Classify each activity as fixed traversal or condition-controlled repetition and justify one classification.

   Answer and explanation: 18.18 and 18.20 use fixed traversal; 18.19, 18.21, and 18.22 are condition-controlled. The justification must point to a known collection/range or an unknown pass count.

2. Choose one activity and list the complete state that must still be correct after the loop ends.

   Answer and explanation: Accept activity-specific final state, including the collected list, first matching number, smallest value, full cooldown sequence, or rounds/backlog state.

Notebook and submission procedure

1. Work in the provided sample and transfer cells; do not delete or recreate them.
2. Run each cell and compare fresh output with the notebook's stated result before revising.
3. Save or download the completed notebook as one `.ipynb` file.
4. Upload that one notebook to the matching Gradescope assignment. Do not export a `.py` file or create a ZIP.

Contract-controlled page • generated from the current VIP activity profiles and stable notebook cell IDs